

Priority Tag–Based File System for User-Driven Storage Optimization

Bhuvan M Acharya
Department of Computer Science and
Engineering
Indian Institute of Information
Technology Tiruchirappalli
India
241114@iiitt.ac.in

Madhav Murari Kartikeya
Department of Computer Science and
Engineering
Indian Institute of Information
Technology Tiruchirappalli
India
241126@iiitt.ac.in

Yogesh
Department of Computer Science and
Engineering
Indian Institute of Information
Technology Tiruchirappalli
India
241161@iiitt.ac.in

Abstract—Traditional file systems primarily rely on hierarchical directory structures for organizing and accessing files, without considering the relative importance of files based on user preferences. This paper presents a Priority Tag File System (PTFS) implemented by modifying the MIT xv6 kernel to introduce a user-driven storage model. In the proposed approach, users assign tags to files and define priority levels for each tag. The file system then allocates disk blocks based on these priorities, ensuring that higher-priority files are placed in faster access regions, while lower-priority files are stored in comparatively slower regions. The xv6 kernel was extended to include tag management, priority mapping, and priority-based block allocation mechanisms. The system dynamically organizes file placement according to user-defined priorities while maintaining compatibility with existing xv6 file operations. Experimental evaluation demonstrates improved access efficiency for high-priority files and enhanced flexibility compared to traditional hierarchical storage methods. The proposed Priority Tag File System provides a user-centric and performance-aware storage approach that adapts to user preferences and access patterns.

Keywords— Priority tag file system, xv6 kernel, priority-based storage, adaptive file system, operating system kernel

I. INTRODUCTION

File systems are a fundamental component of operating systems, responsible for organizing, storing, and retrieving data efficiently. Traditional file systems such as FAT, NTFS, and Unix-based file systems primarily rely on hierarchical directory structures to manage files. While this approach provides logical organization and ease of navigation, it does not consider the relative importance of files based on user preferences or access requirements. As a result, all files are treated uniformly during storage allocation, regardless of their usage frequency or priority, which can lead to inefficient access patterns for frequently used or critical files.

Modern computing environments often involve large volumes of data where certain files require faster access than others. For example, users may frequently access project files, configuration files, or system-critical data, while other files such as archives or media content may be accessed infrequently. Traditional hierarchical file systems do not provide a mechanism for users to explicitly define file importance and influence storage placement accordingly. This limitation motivates the need for a more flexible and user-centric file storage approach.

To address this limitation, this paper proposes a Priority Tag File System (PTFS) implemented by modifying the MIT xv6 kernel. In the proposed system, users assign tags to files

and define priority levels for each tag. Based on these priorities, the file system organizes file storage by placing higher-priority files in faster access regions, while lower-priority files are stored in comparatively slower regions. This approach enables user-controlled storage optimization and improves access efficiency for important files.

The proposed system extends the xv6 file system by incorporating tag management, priority-based block allocation, and dynamic file placement mechanisms. By integrating these features into the xv6 kernel, the Priority Tag File System provides a simple yet effective framework for exploring user-driven storage optimization in operating systems. Experimental evaluation demonstrates that the proposed approach improves access performance for high-priority files while maintaining system simplicity and compatibility with existing file operations.

The main contributions of this work are:

- Priority tag-based file system design
- Weighted priority calculation model
- Dynamic block reordering mechanism
- xv6 kernel modification

The proposed Priority Tag File System introduces a novel, user-centric approach to file storage, offering improved flexibility and performance compared to traditional hierarchical file systems.

II. LITERATURE REVIEW

File system design has evolved significantly to improve storage efficiency, access performance, and scalability. Several existing file systems and storage optimization techniques have explored methods related to performance-aware data placement. This section reviews existing systems and technologies related to the proposed Priority Tag File System.

A. MIT xv6 File System

The MIT xv6 operating system is a Unix-like educational operating system designed to demonstrate core operating system concepts such as process management, memory management, and file systems [1]. The xv6 file system uses a hierarchical directory structure with inode-based file allocation and fixed-size disk blocks. Files are stored sequentially using a traditional block allocation strategy without considering file importance or user-defined priorities.

Although xv6 provides a simple and efficient file system, it lacks mechanisms for prioritizing files based on user requirements or optimizing storage placement for faster access. The proposed Priority Tag File System modifies the xv6 kernel to introduce user-defined priority-based block allocation.

B. Log-Structured File System (LFS)

The Log-Structured File System (LFS), introduced by Rosenblum and Ousterhout, improves file system performance by writing all modifications sequentially to disk [2]. This reduces disk seek time and improves write performance. LFS treats disk storage as a continuous log and periodically cleans old segments.

While LFS improves performance by optimizing disk layout, it does not consider file importance or user-defined priorities. The focus is primarily on improving write throughput rather than optimizing access speed for specific files.

C. EXT4 File System

EXT4 is a widely used Linux file system that improves performance through techniques such as extents, delayed allocation, and multiblock allocation [3]. EXT4 attempts to store files contiguously to improve read performance and reduce fragmentation.

Although EXT4 improves file placement efficiency, it does not provide a mechanism for user-defined file priority or explicit control over block placement based on file importance.

D. Hot and Cold Data Placement (Tiered Storage Systems)

Modern storage systems often classify data into hot and cold categories [4]. Frequently accessed data (hot data) is stored in faster storage devices such as SSDs, while rarely accessed data (cold data) is moved to slower storage such as HDDs.

These systems automatically determine file importance based on access patterns. However, they rely on heuristics and background monitoring rather than explicit user input. In contrast, the proposed Priority Tag File System allows users to directly define priorities.

E. Multi-Tier Storage Systems

Multi-tier storage systems distribute data across multiple storage layers such as cache, SSD, and HDD [5]. Files are assigned priorities and placed in appropriate storage tiers to optimize performance.

While multi-tier storage systems use priority-based placement, they typically operate at hardware or storage controller levels rather than integrating directly into file system block allocation.

III. SYSTEM ARCHITECTURE

The Priority Tag File System (PTFS) is implemented by extending the MIT xv6 kernel with priority-based file allocation mechanisms. The architecture consists of user-level utilities, system call interfaces, priority tag management, and modified file system block allocation.

At the user level, new commands such as `tag`, `addtag`, `listtags`, and `clear` are introduced to allow users to assign tags and define priorities. These utilities communicate with the kernel through newly added system calls integrated into the xv6 system call framework.

The system call layer was extended by modifying `syscall.c`, `syscall.h`, and `sysfile.c` to support priority tag operations. These system calls pass user-defined tag and

priority information to the kernel-level Priority Tag File System module.

The core PTFS logic is implemented in newly added kernel modules `ptfs.c` and `ptfs.h`. These modules manage tag definitions, maintain priority mappings, and determine file placement based on assigned priorities. The PTFS module interacts directly with the xv6 file system.

The file system layer was modified by updating `fs.c`, `fs.h`, `file.c`, and `file.h` to incorporate priority-based block allocation. During file creation and modification, the system determines block placement according to assigned priority levels. Higher-priority files are allocated blocks in faster access regions, while lower-priority files are placed in comparatively slower regions.

This architecture enables seamless integration of user-defined priority tags into the xv6 kernel while maintaining compatibility with existing file system operations. The resulting system provides a flexible, user-centric file storage mechanism that improves access performance for prioritized files.

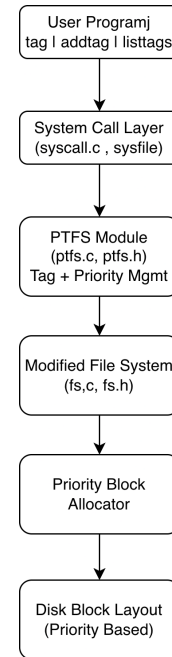


Fig. 1. Priority tag file system architecture

IV.

IMPLEMENTATION DETAILS

The Priority Tag File System (PTFS) was implemented by modifying the MIT xv6 kernel to support priority-based file placement using user-defined tags. The implementation introduces new kernel modules, system calls, and user-level utilities to manage file priorities and dynamically reorder storage blocks.

A. Default File Creation and Priority Assignment

In the modified xv6 kernel, when a new file is created using commands such as ``echo "text" > filename``, the file is automatically assigned a default base priority value of 50. This default priority ensures that newly created files are initially placed in a neutral position within the disk block layout. The priority value is stored in the file metadata and is used during block allocation.

B. Tag Management

The system introduces tag-based priority management using commands such as ``addtag`` and ``removetag``. Users can assign up to five tags to a file, where each tag corresponds to a predefined priority level. When a tag is added or removed, the file system dynamically recalculates the effective priority of the file. The `.config_tag` file contains the priority for each of the tag which can be modified by the user.

Upon any tag modification, the system triggers a reordering mechanism that relocates the file blocks according to the updated priority. Higher priority files are moved toward lower block numbers, while lower priority files are moved toward higher block numbers. Figures and Tables

C. Priority Calculation Model

The Priority Tag File System uses a weighted priority model to determine file placement. Each file is assigned a priority value based on a combination of default priority, tag-based weights, and file access frequency. This approach allows dynamic prioritization based on user preferences and usage patterns.

When a file is created, it is assigned a default base priority of 50. Users can then assign tags to the file using the ``addtag`` command. Each tag has an associated priority value defined in the `.config_tag` configuration file. These tag priorities are retrieved and combined to compute the overall file priority.

In addition to tag-based priorities, the system also incorporates file access frequency. Each time a file is accessed, the access count is incremented, and this value contributes to the overall priority. This ensures that frequently accessed files gradually move to higher priority levels.

The priority of a file is calculated using the following weighted sum:

$$\text{Priority} = \text{Base Priority} + \Sigma(\text{Tag Priorities}) + \text{Access Count}$$

Where:

- Base Priority is the default value assigned during file creation
- Tag Priorities represent weights assigned to user-defined tags
- Access Count reflects file usage frequency

After computing the priority, the value is stored in the inode structure. The updated priority is then used by the reordering mechanism to dynamically adjust file placement within disk blocks. Higher priority files are moved to lower block numbers, ensuring faster access, while lower priority files are placed in higher block regions.

This weighted priority model enables adaptive file placement based on both user-defined preferences and runtime access patterns.

D. Priority-Based Block Reordering

The block allocation strategy was modified to reorder files dynamically based on priority values. The system ensures that:

- Higher priority files are assigned lower block numbers
- Lower priority files are assigned higher block numbers
- File blocks are reorganized automatically when priorities change

This mechanism ensures faster access for higher priority files and improves storage efficiency.

E. Memory Block Visualization

To verify block allocation and priority-based ordering, a debugging command ``dumpblocks`` was implemented. This command prints the entire disk block layout, allowing visualization of file placement across memory blocks.

Using the ``dumpblocks`` command, it can be observed that newly created files appear at the end of the block list. When priorities are modified, files are dynamically reordered such that block numbers decrease with increasing priority.

F. Tag Listing

The ``listtags`` command was implemented to display all assigned tags for a given file. This command retrieves tag metadata and prints associated priority values. This functionality allows users to verify tag assignments and priority levels for files.

G. Dynamic Reordering Mechanism

The system continuously monitors tag changes and dynamically updates file placement. Whenever a tag is added or removed, the file system recalculates priority and reorders blocks accordingly. This dynamic behavior ensures that storage organization always reflects current user preferences.

The implemented Priority Tag File System successfully integrates user-defined priority management into the xv6 kernel while maintaining compatibility with existing file system operations.

The complete source code of the Priority Tag File System is available as an open-source project at: <https://github.com/j4b3-21/Priority-Tag-File-System>

V. TESTING AND RESULTS

A. Dynamic Reordering Mechanism

The modified xv6 kernel was successfully compiled and executed using the QEMU emulator. The system booted correctly and provided access to the shell interface, confirming successful kernel modification and integration of the Priority Tag File System.

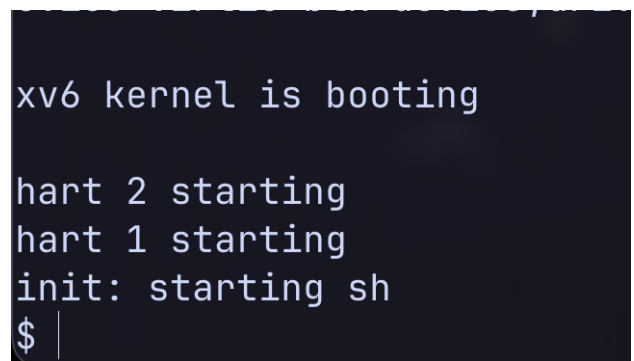


Fig. 2. Modified xv6 kernel booting in QEMU emulator

B. Default File Creation

A file was created using the standard xv6 command. Newly created files were assigned a default base priority value of 50 and adds the access count into the priority. The `dumpblocks` command confirmed that the file was allocated a block based on the default priority.

```
ptfs: block=89 file_block=0 file=/file1 priority=53 tags=[]
ptfs: end block dump
$
```

Fig. 3. Default priority assignment for newly created file

C. Adding Tag

After assigning a high-priority tag using the `addtag` command, the system dynamically reordered file blocks. The file moved to a lower block number, demonstrating priority-based block allocation.

```
ptfs: block=89 file_block=0 file=/file1 priority=133 tags=[important]
ptfs: end block dump
$
```

Fig. 4. File block reordering after priority assignment

D. Multiple File Comparison

Multiple files were created and assigned different priority tags. The `dumpblocks` output shows that higher priority files were allocated lower block numbers, while lower priority files were placed at higher block numbers.

```
ptfs: block=89 file_block=0 file=/file1 priority=143 tags=[important,backup]
ptfs: block=1189 file_block=0 file=/file2 priority=103 tags=[os]
ptfs: block=1190 file_block=0 file=/file3 priority=93 tags=[project]
ptfs: end block dump
$
```

Fig. 5. Priority-based block allocation for multiple files

E. listtags Output

The `listtags` command displays all tags assigned to a file. This confirms that tag metadata is stored and retrieved correctly from the file system.

```
$ listtags file1
important,backup
$ listtags file2
os
$ listtags file3
project
$
```

Fig. 6. Tag listing for prioritized file

The testing results confirm that the Priority Tag File System successfully implements dynamic priority-based block allocation. Files are reordered automatically when priorities change, and block numbers decrease with increasing priority. The system maintains compatibility with existing xv6 operations while providing improved storage organization.

VI. CHALLENGES AND LIMITATIONS

The Priority Tag File System was implemented as a minimal viable prototype by modifying the MIT xv6 kernel. Due to design constraints and limited development time, several compromises were made during implementation.

One major limitation is that the reordering mechanism currently supports only single-block files. Files occupying multiple disk blocks are excluded from reordering to avoid complex block relocation and fragmentation issues. This

constraint simplifies implementation but limits the applicability of priority-based placement for larger files.

Another limitation is the use of linear sorting for file prioritization. The system collects all file inodes and performs a comparison-based sorting algorithm to reorder files. While this approach is sufficient for small-scale environments such as xv6, it may not scale efficiently for large file systems with thousands of files.

The implementation also relies on a global tag configuration file (`.config_tag`) to store tag priorities. While this allows flexible user-defined priority assignment, it introduces dependency on manual configuration and lacks automated priority learning mechanisms.

Additionally, the current implementation supports a maximum of five tags per file, which limits flexibility in complex tagging scenarios. This constraint was introduced to simplify inode structure modifications and reduce metadata overhead.

The system currently performs block swapping instead of full file relocation. While this reduces complexity, it may not fully optimize disk locality for larger files or multi-block data structures.

Furthermore, the prototype implementation does not modify existing scheduler or interrupt mechanisms. The focus of the project was limited to file system modifications, and deeper integration with kernel scheduling or caching subsystems was not explored.

Finally, the Priority Tag File System was tested within the xv6 educational environment, which has limited disk size and simplified file system structures. As a result, performance evaluation in large-scale or real-world environments remains future work.

VII. CONCLUSION

This paper presented the design and implementation of a Priority Tag File System by modifying the MIT xv6 kernel. The proposed system introduces user-defined tags and weighted priority calculation to dynamically reorder file placement within disk blocks. Files with higher priority values are placed in lower block numbers, enabling faster access and improved storage organization.

The implementation successfully integrates tag management, priority calculation, and dynamic block reordering into the xv6 file system. The weighted priority model combines default priority, tag-based weights, and access frequency, allowing adaptive file placement based on user preferences and usage patterns.

Although the current implementation serves as a minimal viable prototype, it demonstrates the feasibility of priority-based file storage within an operating system kernel. Future work includes support for multi-block files, improved sorting algorithms, automated priority learning, and integration with caching and scheduling mechanisms.

The Priority Tag File System introduces a user-centric approach to storage optimization and provides a foundation for further research in adaptive file system design.

ACKNOWLEDGMENT

The authors would like to thank the course instructor for providing guidance and support throughout this project. The MIT xv6 operating system served as the foundational framework for this implementation. The authors also acknowledge the open-source community and available documentation that contributed to the development and testing of the Priority Tag File System.

REFERENCES

1. R. Cox, F. Kaashoek, and R. Morris, "xv6: a simple unix-like teaching operating system," Massachusetts Institute of Technology, 2022. [Online]. Available: <https://pdos.csail.mit.edu/6.828/2025/xv6/book-riscv-rev5.pdf>
2. M. Rosenblum and J. K. Ousterhout, "The design and implementation of a log-structured file system," ACM Transactions on Computer Systems, vol. 10, no. 1, pp. 26–52, Feb. 1992. [Online]. Available: <https://people.eecs.berkeley.edu/~brewer/cs262/LFS.pdf>
3. Linux Kernel Organization, "The ext4 filesystem," Linux Kernel Documentation. [Online]. Available: <https://www.kernel.org/doc/html/latest/filesystems/ext4/index.html>
4. A. Gulati, C. Kumar, and I. Ahmad, "Storage workload classification for tiered storage systems," IBM Research Report, 2009. [Online]. Available: <https://research.ibm.com/publications/storage-workload-classification-for-tiered-storage-systems>
5. A. Verma, R. Koller, L. Useche, and R. Rangaswami, "Srcmap: energy proportional storage using dynamic tiering," Proceedings of the 8th USENIX Conference on File and Storage Technologies, 2010. [Online]. Available: https://www.usenix.org/legacy/event/fast10/tech/full_papers/verma.pdf
6. Priority tag file system project repository. [Online]. Available: <https://github.com/j4b3-21/Priority-Tag-File-System>